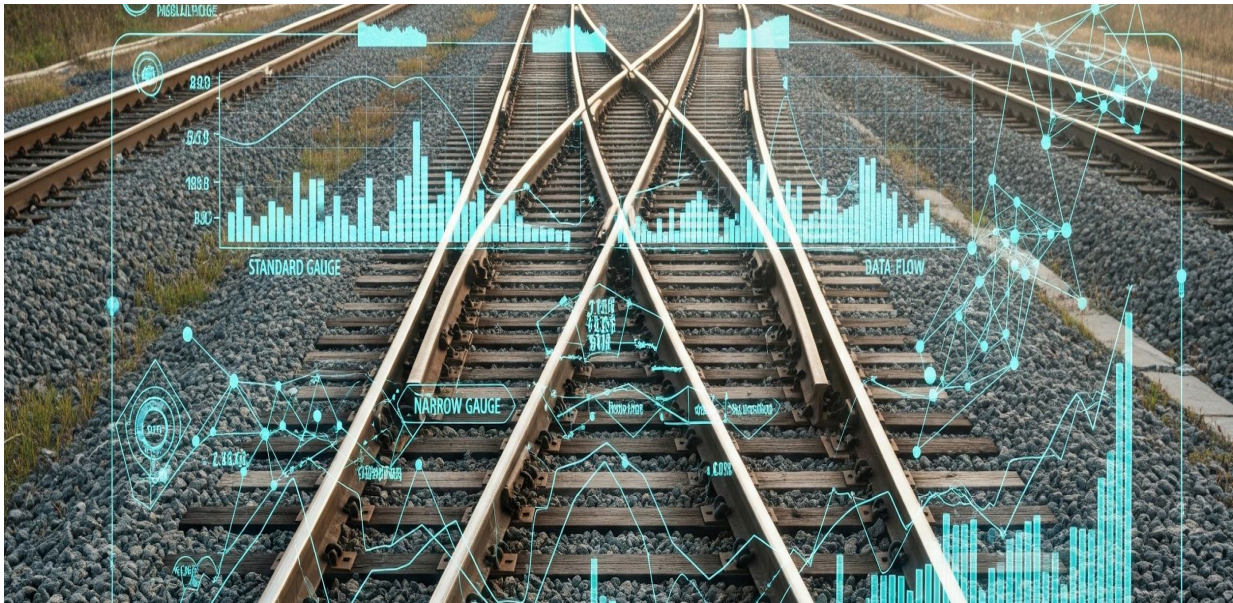




Data Analytics Project: Railway Gauge Trends

Dataset Used: `railway_gauges.csv`

Dataset Type: CSV (Comma-Separated Values)

Dataset Columns: `Year`, `Broad`, `Metre`, `Narrow`, `Total`

This project uses the `railway_gauges.csv` dataset to analyze historical railway gauge data. The dataset contains yearly information about **Broad Gauge**, **Metre Gauge**, **Narrow Gauge**, and **Total railway tracks**. The project uses Python, Pandas, NumPy, and Matplotlib to clean, analyze, visualize, and interpret the railway gauge trends.

Modules Used

The primary libraries/modules used in this project are:

- **Pandas:** Used for loading the CSV file, handling missing values, filtering data, selecting columns, calculating totals, and creating new columns.
- **NumPy:** Used for numerical calculations, especially `np.diff()` to calculate yearly growth in Total tracks.
- **Matplotlib:** Used for creating bar charts, line graphs, pie charts, and stacked bar charts.

Project Coverage

Category	Skills Covered
Data Preprocessing	Data Loading, Missing Value Handling, Type Conversion
Pandas Operations	Filtering, Selection, Aggregation, Feature Engineering
NumPy Calculations	Yearly growth calculation (np.diff)
Visualization	Line Graph, Bar Chart, Pie Chart, Stacked Bar Chart

Scenario 1: Basic Data Loading & Cleaning (● Easy)

Description

In this scenario, the `railway_gauges.csv` dataset is loaded using Pandas. The first five rows and column names are displayed to understand the structure of the dataset. Missing values are checked and replaced with zero. The Broad, Metre, Narrow, and Total columns are then converted into numeric data types so that mathematical calculations and analysis can be performed correctly.

Tasks Completed

1. Loaded the `railway_gauges.csv` dataset into a Pandas DataFrame.
2. Displayed the first 5 rows using `df.head()`.
3. Displayed the column names using `df.columns`.
4. Checked for missing values using `df.isnull().sum()`.
5. Replaced missing values with 0 using `df.fillna(0)`.
6. Converted Broad, Metre, Narrow, and Total columns to numeric types using `pd.to_numeric()`

Task	Description
1.	Load the dataset into a Pandas DataFrame.
2.	Display the first 5 rows and column names to understand structure.
3.	Check for missing values and replace them with 0 (assuming missing values represent 0 km of track).
4.	Convert all relevant gauge columns (Broad, Metre, Narrow, Total) to numeric types (e.g., float64 or int64).

Graph (Optional):

A **bar chart** was created to compare the number of Broad, Metre, and Narrow Gauge railway tracks for each year.

Graph Title:

Gauges: Number of Railway Tracks Installed Per Year

X-axis: Year

Y-axis: Number of Railway Tracks

The graph was saved as:

`rail_gauges.png`

Conclusion:

The railway gauge dataset was successfully loaded and cleaned using Pandas. The first five rows and column names helped in understanding the structure of the dataset. Missing values were replaced with zero, and the gauge-related columns were converted into numeric types. The cleaned dataset was then ready for further analysis and visualization.

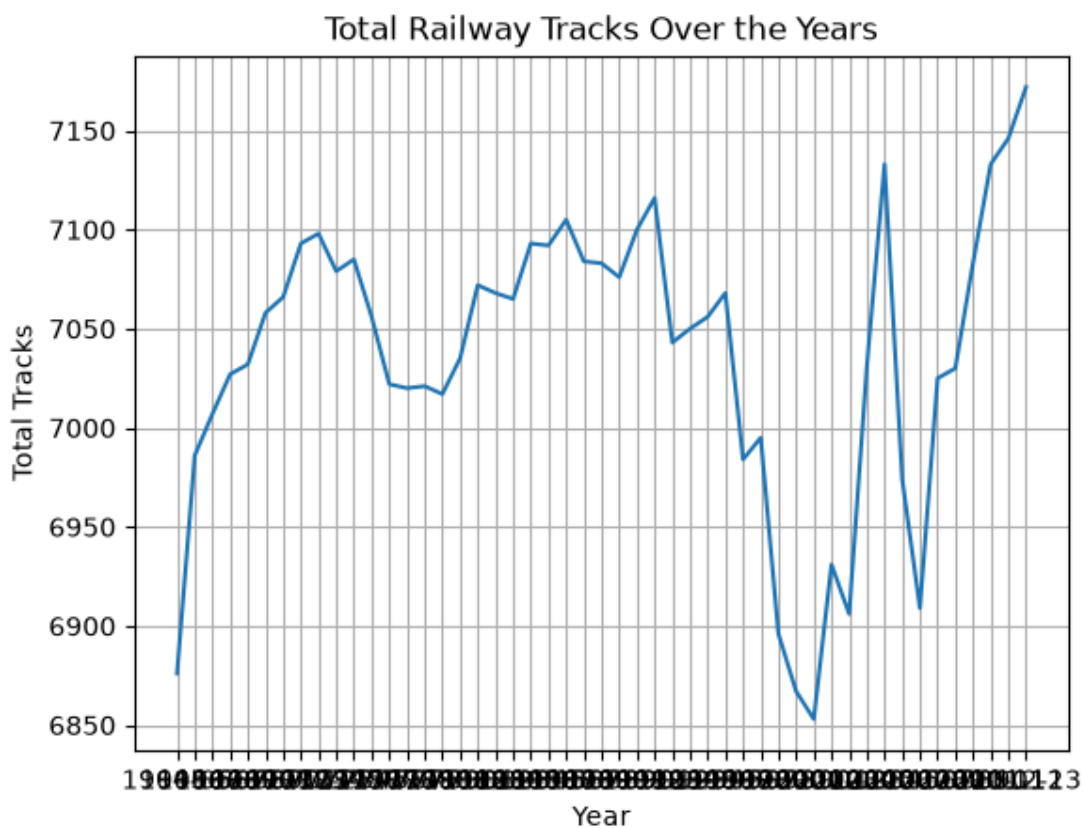
Scenario 2: Simple Visualization (● Easy)

Description

In this scenario, the **Year** and **Total** columns are extracted from the railway gauge dataset. A line graph is created to show the total number of railway tracks over the years. A suitable title and labels are added to make the graph easy to understand. The first and last Total values are compared to identify whether the overall trend is increasing or decreasing

Task	Description
1.	Extracted the Year and Total columns.
2.	Created a line graph and Showing Total tracks over the year
3.	Add a descriptive title X/Y axis labels.
4.	Compared the first and last Total values to identify the overall trend.

Graph (Optional):



Graph: Total Railway Tracks Over the Years

The line graph shows the change in total railway tracks over the years.

X-axis: Year

Y-axis: Total Tracks

Conclusion:

The line graph provides a simple visual representation of the total railway network over time. The first and last Total values are compared to identify the overall direction of the trend. If the final value is greater than the initial value, the trend is considered increasing. This analysis helps understand the overall change in the railway network during the period covered by the dataset.

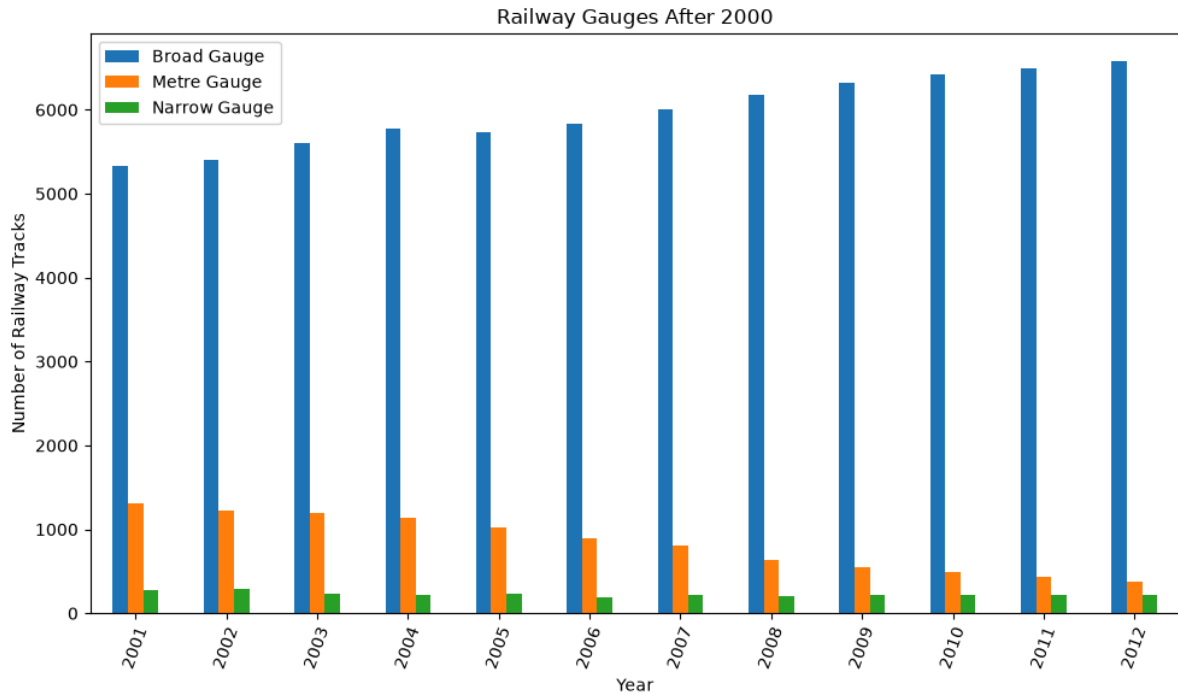
Scenario 3: Filtering + Bar Chart (🟡 Medium)

Description:

In this scenario, the dataset is filtered to include only years after 2000. Broad Gauge, Metre Gauge, and Narrow Gauge values are selected from the filtered data. A grouped bar chart is created to compare the three gauge types for recent years. The total value of each gauge after 2000 is calculated to identify which gauge dominates in the recent period.

Task	Description
1.	Filtered the dataset to include only years after 2000.
2.	Selected Year, Broad, Metre, and Narrow columns.
3.	Created a grouped bar chart comparing the three gauges.
4.	Added a title, axis labels, and legend.
5.	Calculated the total of each gauge after 2000.

Graph (Optional):



Graph Title: Railway Gauges After 2000

The grouped bar chart compares Broad, Metre, and Narrow Gauge for years after 2000.

X-axis: Year

Y-axis: Number of Railway Tracks

Conclusion:

The dataset was filtered to focus on the years after 2000. The grouped bar chart provides a clear comparison between Broad, Metre, and Narrow Gauge during the recent period. The total value of each gauge was calculated, and `idxmax()` was used to identify the gauge with the highest total. This gauge is considered the dominant gauge after 2000.

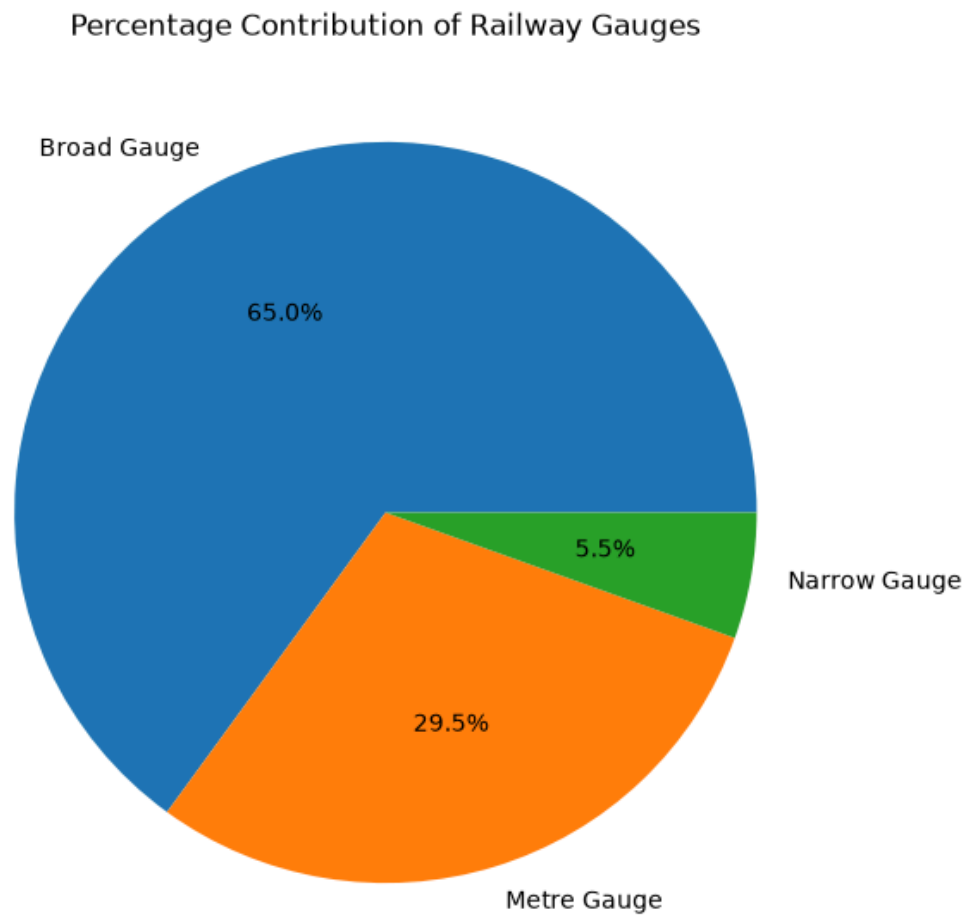
Scenario 4: Feature Engineering + Pie Chart (🟡 Medium)

Description:

In this scenario, the total number of Broad, Metre, and Narrow Gauge tracks across all available years is calculated. These totals are stored in a Pandas Series. A pie chart is then created to show the percentage contribution of each gauge type. The gauge with the highest total contribution is identified using `idxmax()`.

Task	Description
1.	Calculated the total sum of Broad, Metre, and Narrow Gauge across all years.
2.	Stored the three total values in a Pandas Series.
3.	Created a pie chart showing the percentage contribution.
4.	Added percentage labels using <code>autopct</code>
5.	Identified the gauge with the highest contribution.

Graph (Optional):



Graph Title: Percentage Contribution of Railway Gauges

The pie chart represents the overall contribution of:

- Broad Gauge
- Metre Gauge
- Narrow Gauge

The percentage labels show the proportion of each gauge

Conclusion:

The pie chart provides an overall comparison of the three railway gauges across the complete historical period in the dataset. The total of each gauge was calculated using Pandas `sum()`. The gauge with the highest total was identified using `idxmax()`. Therefore, the largest section of the pie chart represents the gauge that contributes the most to the railway gauge data.

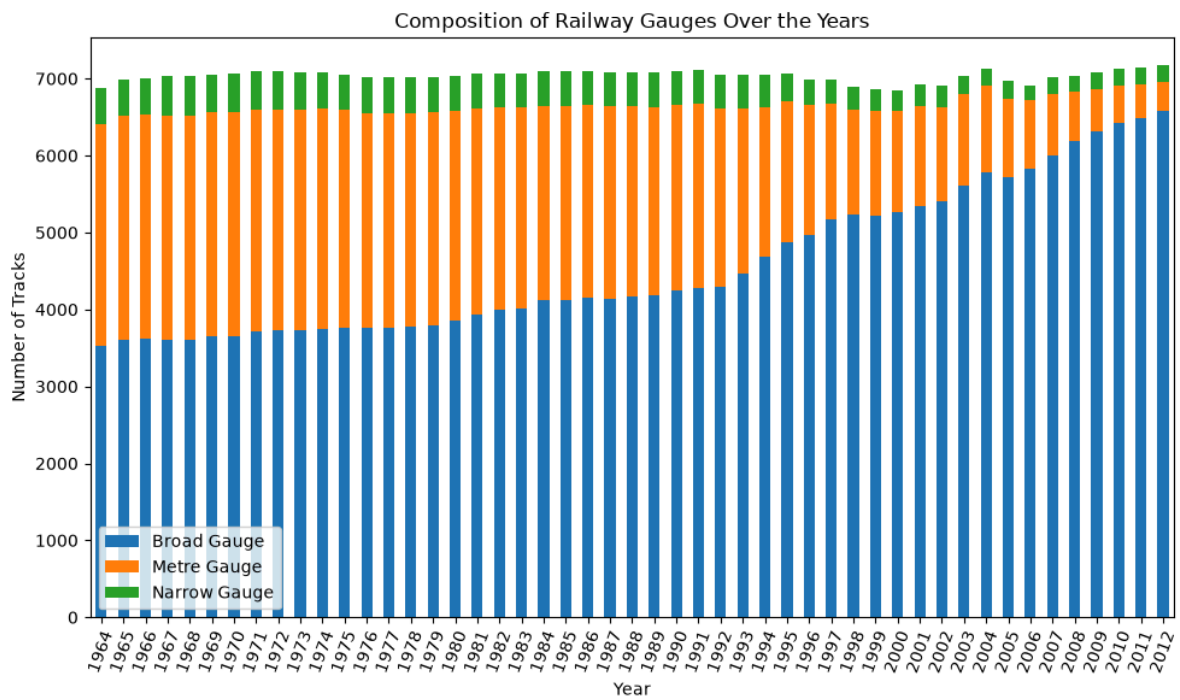
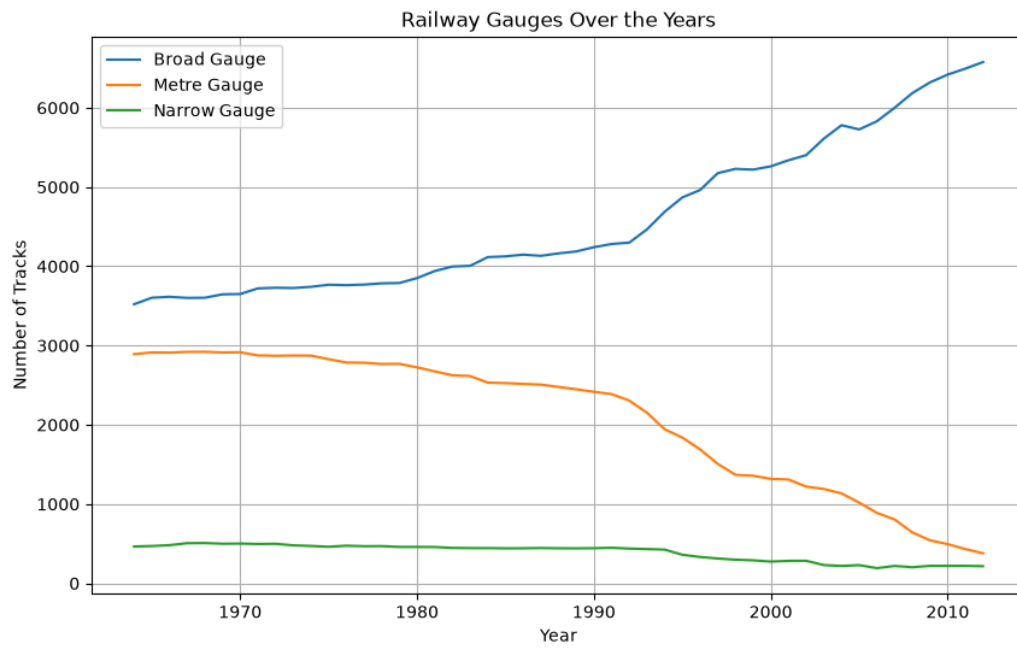
Scenario 5: Advanced Analysis + Multiple Graphs (🔴 Hard)

Description:

In this scenario, advanced analysis is performed on the railway gauge dataset. New percentage columns are created to show the contribution of each gauge relative to Total tracks for every year. NumPy's `np.diff()` function is used to calculate yearly growth in Total tracks. Line and stacked bar charts are created to visualize the railway gauge trends and composition. The analysis also identifies the year with the highest growth and checks whether any gauge experienced a decline.

Task	Description
1.	Created percentage columns for Broad, Metre, and Narrow Gauge relative to Total.
2.	Used <code>np.diff()</code> to calculate yearly growth of Total tracks.
3.	Created a line graph and stacked bar chart.
4.	Identified the year with the highest yearly growth.
5.	Checked whether Broad, Metre, or Narrow Gauge experienced a decline

Graph



Conclusion:

The advanced analysis provides a detailed understanding of railway gauge trends. The percentage columns show the share of each gauge relative to the total railway tracks. The yearly growth calculation identifies the years with the largest changes in the railway network. The decline analysis helps identify periods where individual gauges decreased. The stacked bar chart shows the composition of the railway gauges over time. By examining the percentage trends, it is possible to determine whether the railway system is shifting toward a single dominant gauge.

Final Conclusion:

The Railway Gauge Trends project successfully analyzed historical railway gauge data using Python. The dataset was first loaded and cleaned using Pandas, including handling missing values and converting the gauge columns to numeric types. The railway network trend was then analyzed using line and bar charts.

The data after 2000 was filtered to identify the dominant gauge in recent years. A pie chart was used to understand the overall contribution of Broad, Metre, and Narrow Gauge across all available years. In the advanced analysis, percentage composition and yearly growth were calculated using Pandas and NumPy.

Different graphs helped visualize the railway gauge trends, composition, growth, and decline. The final percentage analysis can be used to determine whether the railway system is gradually shifting toward one dominant gauge.

What you have learned

By completing this project, I learned how to perform data analysis using Python and its important libraries.

I learned how to load a CSV dataset using Pandas and understand its structure using `head()` and `columns`. I learned how to check missing values using `isnull().sum()` and replace missing values using `fillna(0)`.

I learned how to convert data into numeric form using `pd.to_numeric()`. I also learned how to filter a DataFrame using conditions, such as selecting only years after 2000.

I learned how to select particular columns, calculate totals using `sum()`, and find the largest value using `idxmax()`. I also learned how to create new percentage columns using mathematical operations.

Using NumPy, I learned how to use `np.diff()` to calculate the yearly change in Total railway tracks. I also learned how `.diff()` can be used to identify decreases in individual gauge values.